

PROPOSTA DO TRABALHO DE CONCLUSÃO DE CURSO

Victor Leôncio Ribeiro
28 de janeiro de 2025

1 DISPOSIÇÕES GERAIS

As disposições gerais do Trabalho de Conclusão de Curso (TCC), a ser completado na disciplina TCC II, correspondem a:

- Título: Ferramenta de Detecção de *SQL Injection* Utilizando *Word2Vec* e *Random Forest*;
- Modalidade: Acadêmico;
- Orientador: Dalbert Matos Mascarenhas;
- Coordenador: Luís Domingues Tomé Jardim Tarrataca.

2 TEMA

O trabalho tem como tema a detecção de *SQL Injection* utilizando técnicas de aprendizado de máquina e processamento de linguagem natural. Para isso, foi desenvolvido um modelo baseado em *Random Forest*, treinado com um *dataset* composto por comandos SQL normais e injeções SQL maliciosas. O modelo utiliza a representação numérica das frases gerada pelo Word2Vec, uma abordagem que se mostrou eficiente em termos de velocidade e precisão. Além disso, foi criada uma arquitetura com contêineres Docker, integrando um servidor web (*frontend* e *backend*) com um modelo hospedado em FastAPI, permitindo uma detecção automática de ataques e implementação de ações preventivas, como o bloqueio de IPs maliciosos no Iptables. Esse trabalho visa oferecer uma solução robusta e prática para mitigar riscos de segurança em sistemas que utilizam bancos de dados SQL.

3 DELIMITAÇÃO

O objeto de estudo deste trabalho são os comandos SQL normais e maliciosos extraídos de um *dataset* público, que não foi previamente higienizado e balanceado. A proposta limita-se a detectar ataques de *SQL Injection* utilizando aprendizado de máquina, com foco específico no modelo *Random Forest* combinado à representação numérica das frases gerada pelo Word2Vec.

Como restrição, o modelo desenvolvido não aborda outras formas de ataques cibernéticos além do *SQL Injection*, e só analisa dados em tempo real diretamente do tráfego de redes interna e local. A análise foi realizada exclusivamente sobre entradas em formato textual presentes no *dataset* utilizado. Além disso, o modelo é implementado e testado em ambiente controlado com uma arquitetura baseada em Docker, onde a comunicação entre os módulos ocorre via APIs REST. O foco principal está em avaliar a precisão e a velocidade do modelo, bem como sua capacidade de identificar e bloquear IPs maliciosos por meio do uso de ferramentas de firewall como Iptables.

4 JUSTIFICATIVA

Com o aumento da dependência de sistemas informatizados, também crescem as ameaças relacionadas à segurança de dados, sendo os ataques de *SQL Injection* um dos mais frequentes e prejudiciais. Esses ataques exploram vulnerabilidades em aplicações web para obter acesso não autorizado a informações sensíveis e para realizar a exclusão de bancos de dados, causando danos financeiros e reputacionais.

Detectar esses ataques de forma manual é inviável devido à complexidade e ao grande volume de dados que sistemas modernos geram. Apesar de existirem ferramentas de detecção de intrusão, muitas não são acessíveis para empresas de pequeno e médio porte, ou não são otimizadas para esse tipo específico de ataque.

Este trabalho justifica-se pela necessidade de criar uma solução eficiente, acessível e automatizada para detectar *SQL Injection*, contribuindo para aumentar a segurança de sistemas que utilizam bancos de dados. A utilização de aprendizado de máquina, especificamente do modelo *Random Forest* e da técnica Word2Vec, permite a análise rápida e precisa de comandos SQL. Além disso, a implementação de um sistema baseado em Docker possibilita uma integração simples e eficaz em diferentes cenários. Dessa forma, espera-se que este trabalho contribua significativamente para a mitigação de riscos relacionados a ataques de *SQL Injection*.

5 OBJETIVO

O objetivo geral deste trabalho é desenvolver uma ferramenta eficiente e automatizada para a detecção de *SQL Injection* em aplicações web, utilizando técnicas de *Machine Learning*, especificamente *Random Forest*, em combinação com a representação numérica de dados textuais via Word2Vec. A proposta busca aumentar a segurança de sistemas contra ameaças relacionadas à exploração de vulnerabilidades em consultas SQL, proporcionando uma solução

acessível e de alta performance para prevenir ataques.

Com isso, espera-se não apenas identificar ataques com maior precisão e rapidez, mas também implementar uma arquitetura que integre essa solução ao ambiente operacional, bloqueando automaticamente endereços IP suspeitos e fornecendo métricas que possam ser utilizadas por analistas de segurança para avaliar e mitigar riscos futuros.

METODOLOGIA

A metodologia para o desenvolvimento da ferramenta foi estruturada em várias etapas que englobam desde a coleta e preparação dos dados até a implementação de um sistema completo para detecção de SQL Injection. Essas etapas foram:

1. Coleta e Higienização do Dataset:

O primeiro passo foi trabalhar com um *dataset* que continha frases SQL e SQL Injection, divididas em duas classes: SQL normal (0) e SQL Injection (1). Realizou-se uma limpeza e higienização dos dados, removendo ruídos e inconsistências.

2. Vetorização das Frases:

As frases textuais foram convertidas em representações numéricas para que pudessem ser processadas pelo modelo de *machine learning*. Duas técnicas foram avaliadas:

- **TF-IDF**: Utilizado como abordagem inicial, baseado na frequência de termos.
- **Word2Vec**: Escolhido após testes comparativos devido à sua maior eficiência e rapidez durante o treinamento do modelo.

3. Balanceamento do Dataset:

Foi observado que as classes no *dataset* estavam desbalanceadas, o que poderia afetar o desempenho do modelo. Para resolver isso:

- Utilizou-se o algoritmo **SMOTE** (*Synthetic Minority Over-sampling Technique*), que cria exemplos sintéticos para equilibrar as classes.
- Posteriormente, também foi testada a estratégia de ajuste do parâmetro `class_weight` no modelo *Random Forest*, sendo mais interessante que o SMOTE.

4. Treinamento do Modelo:

Várias abordagens de treinamento foram realizadas para otimizar o desempenho do modelo:

- Inicialmente, utilizou-se *Random Forest* com TF-IDF e SMOTE, sem seleção de *features*.
- Em seguida, foram ajustados parâmetros como `max_depth` e o número de árvores na floresta (`n_estimators`).
- Implementou-se a técnica de *Feature Selection* com a importância das *features* geradas pelo *Random Forest*.

- Por fim, foi aplicado Word2Vec com o ajuste do parâmetro `class_weight`, o que gerou os melhores resultados.

5. Desenvolvimento da Arquitetura:

Uma arquitetura baseada em contêineres Docker foi desenvolvida para testar e validar o modelo. A solução consistiu em dois contêineres:

- Um servidor web com *Flask*, responsável pelo *frontend* e *backend*.
- Um contêiner separado para o modelo de *machine learning*, acessado via *FastAPI*.

A comunicação entre os contêineres permitiu a integração fluida entre o sistema e o modelo.

6. Implementação de Bloqueio de IP:

Foi implementada uma funcionalidade para adicionar o IP de atacantes detectados ao *iptables* ou outro *firewall*, bloqueando automaticamente acessos maliciosos ao sistema.

7. Avaliação de Desempenho:

Métricas de desempenho, velocidade e tempo de execução foram coletadas em todas as etapas, permitindo avaliar a eficiência e a eficácia do modelo e da arquitetura desenvolvida.

6 MATERIAIS

Para a realização da ferramenta proposta, foi necessário utilizar um ambiente de desenvolvimento com containers Docker para simular um cenário de detecção de SQL Injection. O ambiente foi configurado com dois containers principais: um container Flask, que serviu como servidor web responsável pelo frontend e backend, e outro container FastAPI, que abrigou o modelo de Machine Learning. O sistema operacional utilizado nos containers foi o Ubuntu.

Para o desenvolvimento e treinamento do modelo, foram utilizados os seguintes materiais:

- Um dataset contendo frases SQL normais e com SQL Injection, obtido de fontes públicas.
- Bibliotecas Python, incluindo *scikit-learn*, *gensim* para a vetorização com Word2Vec, e *imbalanced-learn* para balanceamento de dados com SMOTE.
- Um sistema de contêineres Docker para o gerenciamento de aplicação e isolamento da arquitetura.
- Um sistema de firewall para bloquear endereços IP suspeitos, implementado com *iptables*.
- Ferramentas de benchmarking para medir o desempenho do modelo em termos de velocidade, acurácia e tempo de resposta em diferentes etapas do processo.

7 CRONOGRAMA

Figura 7.1: Cronograma TCC

FASES	MESES																	
	fev./24	mar./24	abr./24	maio/24	jun./24	jul./24	ago./24	set./24	out./24	nov./24	dez./24	jan./25	fev./25	mar./25	abr./25	maio/25	jun./25	jul./25
1				Greve	Greve	Férias												
2				Greve	Greve	Férias												
3				Greve	Greve	Férias												
4				Greve	Greve	Férias												
5				Greve	Greve	Férias												
6				Greve	Greve	Férias												
7				Greve	Greve	Férias												
8				Greve	Greve	Férias												
9				Greve	Greve	Férias												

- Fase 1 : Pesquisa e identificação de trabalhos relacionados - Fev/2024 até Março/2024
- Fase 2 : Criação do ambiente para o treinamento do modelo - Abril/2024
- Fase 3 : Realizando o desenvolvimento dos treinamentos - Agosto/2024 até Novembro/2024
- Fase 4 : Criação da arquitetura baseada em contêineres Docker - Dezembro/2024 até Janeiro/2025
- Fase 5 : Início da escrita - Janeiro/2025 até Fevereiro/2025
- Fase 6 : Identificação e consolidação dos resultados - Março/2025 até Abril/2025
- Fase 7 : Melhorias no trabalho e projeção de trabalhos futuros - Abril/2025 até Maio/2025
- Fase 8 : Término da escrita - Maio/2025 até Junho/2025
- Fase 9: Apresentação TCC - Julho/2025

REFERÊNCIAS

- [1] LI, Qi and LI, Weishi and WANG, Junfeng and CHENG, Mingyu. "A SQL Injection Detection Method based on Adaptive Deep Forest". Beijing University of Posts and Telecommunications, Beijing, China; Sichuan University, Chengdu, China, 2019.
- [2] ROY, Prince and GUJRAL, Rajneesh Kumar and RANI, Pooja. "SQL Injection Attack Detection by Machine Learning Classifier". In: 2022 International Conference on Applied Artificial Intelligence and Computing (ICAAIC), May 2022. DOI: 10.1109/ICAAIC53929.2022.9792964.

- [3] PERALTA-GARCIA, Edwin and QUEVEDO-MONSALBE, Juan and TUESTA-MONTEZA, Victor and ARCILA-DIAZ, Juan. "Detecting Structured Query Language Injections in Web Microservices Using Machine Learning". Escuela de Ingeniería de Sistemas, Universidad Señor de Sipán, Chiclayo 14000, Peru, 2024.
- [4] LI, Yingbo and ZHANG, Bin. "Detection of SQL Injection Attacks Based on Improved TFIDF Algorithm". Journal of Physics: Conference Series, vol. 1395, p. 012013, November 2019. DOI: 10.1088/1742-6596/1395/1/012013.
- [5] ZHOU, Jing and YE, Zhanliang and ZHANG, Sheng and GENG, Zhao and HAN, Ning and YANG, Tao. "Investigating response behavior through TF-IDF and Word2vec text analysis: A case study of PISA 2012 problem-solving process data". 2024.
- [6] CUTLER, Adele and CUTLER, David and STEVENS, John. "Random Forests". In: Machine Learning - ML, vol. 45, pp. 157-176, 2011. ISBN: 978-1-4419-9325-0. DOI: 10.1007/978-1-4419-9326-7_5.
- [7] HUSSAIN, Syed Saqlain. "SQL Injection Dataset". Kaggle, 2023. Disponível em: <https://www.kaggle.com/datasets/syedsaqlainhussain/sql-injection-dataset>. Acesso em: 29-02-2024.
- [8] RAD, Babak Bashari and BHATTI, Harrison John and AHMADI, Mohammad. "An Introduction to Docker and Analysis of its Performance". IJCSNS International Journal of Computer Science and Network Security, vol. 17, no. 3, March 2017. Asia Pacific University of Technology and Innovation, Technology Park Malaysia, Kuala Lumpur, Malaysia.